A

Project Report

On

"8-Bit Mini Processor Design and Simulation"

Submitted in Partial Fulfillment of the Requirements for the

Award of the Degree of

Master of Technology (M.Tech)

in

Data Science

Subject: Digital Electronics & Computer Organization (PCS1001)

Submitted by

Abhishek Saxena (Roll No. 2020610101070005)

Under the Guidance of

Mr. Prabhat Srivastava

Assistant Professor

Department of Computer Science & Information Systems



Shri Ramswaroop Memorial University

Lucknow – Deva Road, Barabanki (UP)

Academic Session 2026–2027

DECLARATION

I hereby declare that the project report entitled **"8-Bit Mini Processor Design and Simulation"** submitted by me to **Shri Ramswaroop Memorial University, Lucknow – Deva Road, Barabanki (UP)** is the partial requirement for the award of the degree of Master of Technology in Computer Science, and is a record of bonafide project work for the subject Digital Electronics & Computer Organization (PCS1001) carried out by me under the guidance of **Mr. Prabhat Srivastava, Assistant Professor**. I further declare that the work reported in this project has not been submitted and will not be submitted either in part or in full for the award of any other degree in this institute.

Place: Barabanki (UP)

Date:

Signature of student

(Abhishek Saxena)
(Roll No. 2020610101070005)

ACKNOWLEDGEMENT

The satisfaction that accompanies the successful completion of any task would be incomplete without the mention of the people whose ceaseless cooperation made it possible, whose constant guidance and encouragement crown all efforts with success. I owe many thanks to the many people who assisted and helped me during and till the end of the project.

I would like to express my gratitude towards **Mr. Atul Verma, Head of Department – Computer Science & Information Systems, Shri Ramswaroop Memorial University, Lucknow – Deva Road, Barabanki (UP)**, for his guidelines and scholarly encouragement.

I am indebted to **Mr. Prabhat Srivastava – Assistant Professor, Computer Science & Information Systems** of **Shri Ramswaroop Memorial University, Lucknow – Deva Road, Barabanki (UP)**, faculty for Digital Electronics & Computer Organization (PCS1001), for teaching the subject concepts and providing the academic direction relevant to this project, and for his valuable comments and suggestions that have helped me to make it a success.

I am thankful for the concepts covered in Digital Electronics & Computer Organization, which provided the foundation for understanding processor architecture, registers, arithmetic and logic operations, memory organisation, instruction encoding, and control sequencing.

I gratefully acknowledge and express my gratitude to all the faculty members of the Department of Computer Science & Information Systems, and to my classmates and peers, who supported me in preparing this project report on "8-Bit Mini Processor Design and Simulation." I also acknowledge the textbooks and technical learning resources listed in the references, which were consulted to develop the architecture and simulation model described in this report.

Finally, this acknowledgement is incomplete without extending my deepest-felt thanks and gratitude towards my family members, whose encouragement has been the source of nourishment for me at each stage of this work.

Abhishek Saxena
(Roll No. 2020610101070005)

ABSTRACT

This project presents the design and functional simulation of a small 8-bit mini processor intended to demonstrate the fundamental concepts of digital electronics and computer organisation. The processor follows a simple accumulator-based architecture. It contains an 8-bit accumulator, an 8-bit instruction register, a 4-bit program counter, a control unit, an arithmetic logic unit (ALU), a 16 x 8-bit program memory, a 16 x 8-bit data memory, status flags, and an output register.

Each instruction is encoded in one byte. The upper four bits identify the operation, while the lower four bits carry an immediate value or a memory address. The instruction set includes data transfer, arithmetic, logical, branch, output, and halt operations. The control unit performs the fetch-decode-execute sequence and generates the control actions required by each instruction.

A reference software simulator is used to validate instruction execution. A demonstration program stores the value 5 in data memory, adds 3 to it, stores and reloads the result, subtracts 5, and writes the final value to the output register. The observed output is 3 (0x03). Additional tests cover carry generation, subtraction, logical operations, conditional branching, and halt behaviour. All six verification checks pass in the reference model.

The project is a functional educational model. It demonstrates architectural behaviour and instruction semantics; it does not claim gate-level timing verification, FPGA implementation, or fabrication of a physical processor.

Keywords: 8-bit processor, accumulator, ALU, instruction set, control unit, program counter, functional simulation.

TABLE OF CONTENTS

DECLARATION	**ii**
ACKNOWLEDGEMENT	**iii**
ABSTRACT	**iv**
CHAPTER 1: INTRODUCTION	**1**
1.1 Introduction of the Project	1
1.2 Objective and Scope of the Project	1
1.3 Project Motivation	2
1.4 Problem Statement	2
1.5 Design Approach	2
CHAPTER 2: LITERATURE REVIEW	**3**
2.1 Overview of the 8-Bit Mini Processor	3
2.2 System Specifications	4
CHAPTER 3: DESIGN OF PROJECT MODEL	**5**
3.1 Processor Architecture and Block Diagram	5
3.2 Functional Units	6
3.3 Instruction Set and Encoding	6
3.4 ALU Design and Status Flags	7
3.5 Control Unit and Instruction Cycle	8
3.6 Memory and Input/Output Organisation	9
CHAPTER 4: EXPERIMENTS, SIMULATION & TESTING	**11**
4.1 Methodology	11
4.2 Hardware & Software Used	11
4.3 Testing Technology Used	12
4.4 Demonstration Program	12
CHAPTER 5: RESULT AND DISCUSSION	**13**
5.1 Simulation Trace and Results	13
5.2 Verification and Test Cases	13
5.3 Applications and Advantages	14
CHAPTER 6: CONCLUSION AND FUTURE SCOPE	**16**
6.1 Conclusion	16
6.2 Future Scope	16
REFERENCES	**18**
APPENDIX A: INSTRUCTION ENCODING AND SIMULATOR	**19**

CHAPTER 1
INTRODUCTION

1.1 Introduction of the Project

A processor is the central unit of a computer system that fetches instructions, interprets them, and performs the operations needed to process data. At a basic level, a processor combines a datapath, which stores and transforms values, with a control unit, which sequences the operations.

An 8-bit processor operates on data values that are eight bits wide. Although modern general-purpose processors commonly use wider data paths, an 8-bit design is useful for learning how binary instructions, registers, memory, arithmetic circuits, and control signals work together. A compact processor also makes it possible to trace the execution of every instruction by hand.

1.2 Objective and Scope of the Project

1.2.1 Objectives

- To understand the basic organisation of an 8-bit CPU and the relationship between the datapath and control unit.
- To define the processor registers, memory organisation, instruction format, and a compact instruction set.
- To describe the function of the ALU for arithmetic and bitwise operations.
- To explain the fetch-decode-execute sequence and the role of control signals.
- To execute a demonstration program and inspect the changes in the accumulator, memory, output register, and status flags.
- To verify important instruction behaviours using repeatable functional test cases.

1.2.2 Scope

The design covers a single-accumulator 8-bit processor with a 4-bit program counter, 16 instruction locations, and 16 data locations. The instruction set is intended for demonstration rather than compatibility with an existing commercial processor. The software model is used to check instruction-level behaviour.

1.2.3 Assumptions and Boundaries

- All data values and ALU results are 8-bit values; arithmetic wraps around modulo 256.

- The program ROM and data RAM each contain 16 locations of 8 bits.
- The program counter and address operands are 4 bits wide, so instruction and data addresses range from 0 to 15.
- The instruction format is fixed at 8 bits: opcode[7:4] and operand[3:0].
- The reference simulation is functional and instruction-level; physical gate delays and clock timing are outside the present scope.

1.3 Project Motivation

The project focuses on a deliberately small architecture rather than a commercial processor. The limited memory size and short instruction format keep the design understandable while still demonstrating the essential functions of a CPU: instruction fetch, decode, data movement, arithmetic and logic, conditional control flow, and output.

1.4 Problem Statement

The objective is to specify a consistent 8-bit mini processor and demonstrate that its instruction set can execute a short program correctly. The processor must be able to load constants, read and write data memory, perform arithmetic and bitwise operations, change the program flow using branch instructions, and stop when a halt instruction is encountered.

1.5 Design Approach

A simple accumulator-based datapath is selected. One 8-bit accumulator acts as the primary working register. An instruction byte is divided into a 4-bit opcode and a 4-bit operand. Separate program and data memories are used so instruction fetch and data access are conceptually distinct. A reference simulator executes the defined instruction semantics and records the register and memory values after each instruction.

CHAPTER 2
LITERATURE REVIEW

This chapter presents the background concepts and the architectural overview on which the processor design is based. The concepts follow the computer organisation and digital design textbooks listed in the References [1]–[4].

2.1 Overview of the 8-Bit Mini Processor

The processor uses an accumulator-based architecture. Most arithmetic and logical instructions read the current accumulator value and a second operand from data memory, process both values in the ALU, and write the result back to the accumulator. This reduces the number of registers required and makes instruction execution straightforward to demonstrate.

2.1.1 Main Architectural Blocks

- Program Counter (PC): a 4-bit register containing the address of the next instruction.
- Program ROM: stores up to 16 instruction bytes.
- Instruction Register (IR): holds the instruction currently being decoded and executed.
- Control Unit: decodes the opcode and coordinates register transfers, memory access, ALU selection, and branching.
- Accumulator (ACC): an 8-bit working register used by arithmetic and logical instructions.
- ALU: performs addition, subtraction, AND, OR, and XOR operations.
- Data RAM: stores 16 bytes of program data.
- Status Flags: the zero flag (Z) indicates a zero result; the carry/borrow flag (C) records arithmetic overflow or borrow according to the operation.
- Output Register (OUT): captures the accumulator value when an OUT instruction is executed.

2.1.2 Why an Accumulator Architecture?

A multi-register processor requires register selection and more complex instruction encoding. The accumulator design keeps the datapath compact: one operand is already in ACC and the second operand is selected from memory. This is appropriate for a teaching model because the data flow can be followed with a small number of state elements.

2.1.3 Processor Operation in One Sentence

The PC selects an instruction from program memory, the control unit decodes it, the datapath performs the required action, and the processor updates its registers and flags before moving to the next instruction.

2.2 System Specifications

The following table defines the architectural configuration used throughout this report. The sizes are design choices for this educational mini processor, not specifications of a commercial CPU.

Table 2.1: System specifications of the 8-bit mini processor

Parameter	Specification	Purpose
Data width	8 bits	Width of accumulator, ALU operands, memory words, and output.
Instruction width	8 bits	One-byte instruction containing opcode and operand fields.
Opcode field	4 bits	Selects one of 16 possible instruction classes.
Operand field	4 bits	Immediate value or address from 0 to 15.
Program Counter	4 bits	Addresses 16 instruction locations.
Program memory	16 x 8 bits	Stores the instruction sequence.
Data memory	16 x 8 bits	Stores operands and intermediate data.
Accumulator	8 bits	Primary arithmetic and logical working register.
Flags	Z, C	Zero and carry/borrow status indicators.
Output register	8 bits	Holds the value transferred by OUT.
Execution style	Accumulator-based	One main working register simplifies datapath control.

2.2.1 Data Representation

All values are stored in binary. Hexadecimal notation is used in the report because one hexadecimal digit represents four bits. Therefore, an 8-bit value is written using two hexadecimal digits. For example, decimal 5 is 00000101 in binary and 0x05 in hexadecimal.

2.2.2 Addressing

An instruction operand is four bits wide. For LDA, STA, ADD, SUB, AND, OR, and XOR, the operand is interpreted as a data-memory address from 0 to 15. For LDI, the operand is interpreted as a zero-extended 4-bit immediate value. For JMP, JZ, and JNZ, the operand is interpreted as a program-memory address from 0 to 15.

CHAPTER 3
DESIGN OF PROJECT MODEL

3.1 Processor Architecture and Block Diagram

Figure 3.1 presents the main blocks and the direction of information flow. Program memory supplies the instruction byte. The PC identifies the current instruction location, while the IR holds the fetched instruction. The control unit interprets the opcode and issues the control actions needed by the datapath.

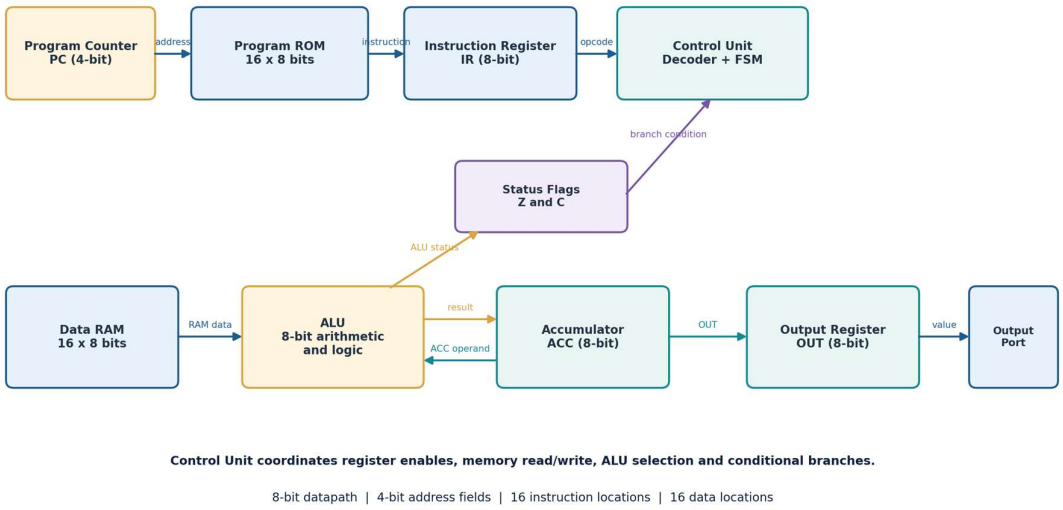


Figure 3.1: Top-level block diagram of the proposed 8-bit mini processor.

3.1.1 Datapath and Control Separation

The datapath contains the accumulator, ALU, data memory interface, and output register. It moves and transforms 8-bit values. The control unit contains the instruction decoder and a finite-state sequence controller. It determines when the PC increments, when a memory location is read or written, which ALU operation is selected, and whether a branch changes the PC.

3.1.2 Information Paths

- Instruction path: PC -> Program ROM -> IR -> Opcode decoder.
- Operand path: IR operand -> address selection -> Data RAM.
- Arithmetic path: ACC and RAM data -> ALU -> ACC.
- Output path: ACC -> OUT register -> output port.
- Status path: ALU result -> Z and C flags -> conditional branch decision.

3.2 Functional Units

3.2.1 Program Counter (PC)

The PC is a 4-bit register that stores the address of the next instruction. During a normal fetch, the selected instruction is read and the PC increments by one. A JMP instruction replaces the PC with the operand. JZ and JNZ replace the PC only when their respective flag condition is true. Since the PC is four bits wide, incrementing past address 15 wraps to address 0.

3.2.2 Instruction Register (IR)

The IR stores the 8-bit instruction currently under execution. Its upper nibble is sent to the opcode decoder and its lower nibble provides the immediate value or address field. Holding the instruction in the IR keeps the instruction data stable while the control unit performs the associated operation.

3.2.3 Accumulator (ACC)

ACC is the primary 8-bit working register. LDI and LDA load it with a value. ADD, SUB, AND, OR, XOR, INC, and DEC update it. STA copies it to data memory, while OUT copies it to the output register.

3.2.4 Data Memory

Data RAM contains 16 addressable bytes. The address operand selects one byte. A read operation supplies the selected value to the ALU or accumulator input. A write operation stores the accumulator value at the selected address.

3.2.5 Output Register

The OUT register is an 8-bit latch that captures the current ACC value when the OUT instruction is executed. It provides a stable result for observation in the functional simulator or a future display/LED interface.

3.2.6 Control Unit

The control unit decodes the instruction and enables the appropriate datapath action. In a hardware implementation, the control signals would drive register enables, memory read/write controls, ALU selection, PC loading, and output capture.

3.3 Instruction Set and Encoding

Every instruction is one byte long. The most significant four bits identify the operation and the least significant four bits carry the operand. This fixed-length format simplifies instruction fetch and decode.

Table 3.1: Instruction format

Bits 7-4	Bits 3-0	Meaning
Opcode	Operand	Operation selected by the opcode; operand is immediate data, memory address, or branch target.

Example: 0x15 is binary 0001 0101. The opcode 0001 selects LDI and the operand 0101 is the immediate value 5. Therefore, the instruction loads 0x05 into ACC.

3.3.1 Defined Instruction Set

Table 3.2: Defined instruction set

Opcode	Mnemonic	Operation
0000	NOP	No operation; processor state is unchanged.
0001	LDI k	Load the 4-bit immediate value k into ACC.
0010	LDA a	Load ACC from data memory address a.
0011	STA a	Store ACC at data memory address a.
0100	ADD a	ACC <- ACC + RAM[a]; update Z and carry.
0101	SUB a	ACC <- ACC - RAM[a]; update Z and borrow.
0110	AND a	ACC <- ACC AND RAM[a]; update Z.
0111	OR a	ACC <- ACC OR RAM[a]; update Z.
1000	XOR a	ACC <- ACC XOR RAM[a]; update Z.
1001	JMP a	Unconditional jump to program address a.
1010	JZ a	Jump to a if Z = 1.
1011	JNZ a	Jump to a if Z = 0.
1100	INC	Increment ACC by one; update Z and carry.
1101	DEC	Decrement ACC by one; update Z and borrow.
1110	OUT	Copy ACC to the output register.
1111	HLT	Stop instruction execution.

The instruction set uses all 16 opcode values. Instructions that do not require an operand, such as NOP, INC, DEC, OUT, and HLT, ignore the low four bits; they are written with an operand of 0 for clarity.

3.4 ALU Design and Status Flags

The ALU is an 8-bit combinational unit. It receives the current accumulator value and a second 8-bit operand from data memory. The control unit selects the required operation. The result is written back to ACC for arithmetic and logical instructions.

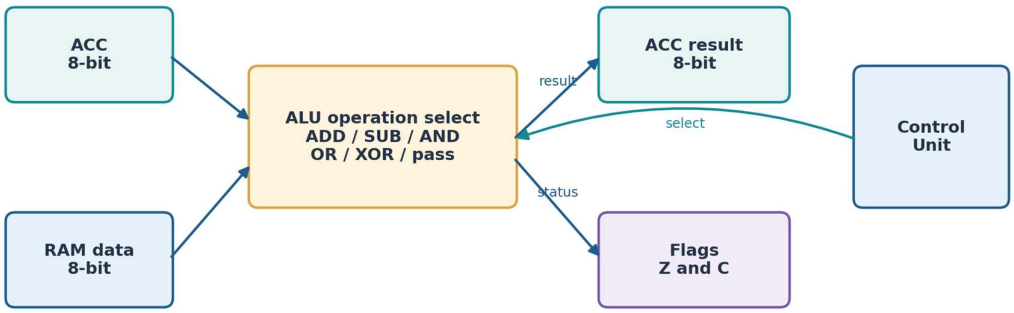


Figure 3.2: Conceptual ALU datapath and status-flag outputs.

3.4.1 Arithmetic Operations

For addition, the 9-bit internal sum is conceptually formed from ACC + operand. The lower eight bits become the result and the ninth bit is captured as carry. For subtraction, the 8-bit result is ACC - operand modulo 256; the C flag is set when the unsigned operation requires a borrow.

3.4.2 Logical Operations

AND, OR, and XOR operate independently on each corresponding bit of the two operands. These instructions do not modify the C flag in the reference model. They update the Z flag according to whether the resulting byte is zero.

3.4.3 Status Flags

Table 3.3: Status flags of the processor

Flag	Meaning	Update rule
Z (Zero)	Indicates whether the latest result is 0.	Z = 1 when ACC = 0; otherwise Z = 0 after instructions that write ACC.
C (Carry/Borrow)	Records unsigned carry for addition or borrow for subtraction.	ADD/INC: set if result exceeds 255. SUB/DEC: set if a borrow is required.

For operations that do not define a carry/borrow outcome, the reference model leaves C unchanged. This rule is stated to make the simulation deterministic and repeatable.

3.5 Control Unit and Instruction Cycle

The control unit coordinates the processor through a repeating fetch-decode-execute sequence. Each instruction passes through the same conceptual stages, although the exact datapath actions depend on the opcode.

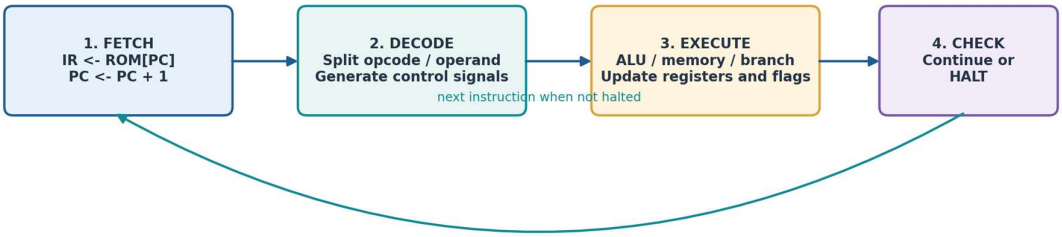


Figure 3.3: Conceptual fetch-decode-execute sequence.

3.5.1 Fetch

The PC supplies the program-memory address. The instruction byte at that address is transferred to the IR. The PC is then incremented so it points to the next sequential instruction.

3.5.2 Decode

The decoder separates the opcode and operand. The control unit determines whether the instruction needs a memory read, memory write, ALU operation, branch evaluation, output transfer, or halt action.

3.5.3 Execute

The datapath performs the selected operation. For example, ADD reads RAM at the operand address, adds the value to ACC, writes the result back to ACC, and updates Z and C. STA writes ACC into data RAM. JMP loads the PC with the target address.

3.5.4 Halt

When HLT is decoded, the processor sets its halted state. No further instruction is executed by the reference simulator unless the processor is reset and a new run is started.

3.6 Memory and Input/Output Organisation

The proposed model uses a simple Harvard-style arrangement with separate program and data memories. Program memory stores instructions, while data RAM stores values processed by the instructions. This separation makes the design easier to explain and avoids using a data write operation to modify the active instruction sequence.

Table 3.4: Memory and register organisation

Address range	Storage	Width	Use
0x0 - 0xF	Program ROM	16 x 8 bits	Stores up to 16 instruction bytes.
0x0 - 0xF	Data RAM	16 x 8 bits	Stores operands and intermediate values.
Internal	ACC / IR / OUT	8 bits each	Working value, current instruction, and observed output.
Internal	PC / operand	4 bits	Instruction sequencing and address selection.
Internal	Z / C	1 bit each	Branch condition and arithmetic status.

3.6.1 Data Transfer Instructions

LDI loads a small immediate constant directly into the accumulator. LDA reads a byte from data RAM. STA writes the accumulator to data RAM. These three instructions establish the basic data movement needed by a small program.

3.6.2 Output Behaviour

OUT copies ACC into the output register. In an extended hardware project, the output register could drive LEDs, a seven-segment display through a decoder, or a simple output port. Those interfaces are not part of the current functional model.

CHAPTER 4
EXPERIMENTS, SIMULATION & TESTING

4.1 Methodology

The processor is validated using a small reference software simulator. The simulator represents the architectural registers and memories as program state. It fetches one instruction byte, decodes the opcode and operand, performs the defined operation, updates the applicable flags, and records a trace row.

4.1.1 Simulation Sequence

- Initialise program memory and data memory with the chosen instruction sequence and input values.
- Set PC to 0, initialise ACC and OUT, and initialise the status flags.
- Fetch the instruction at the PC and increment PC modulo 16.
- Decode the upper nibble as the opcode and the lower nibble as the operand.
- Execute the operation and update ACC, memory, OUT, PC, and flags as required.
- Record the resulting state, then continue until HLT is executed.

4.1.2 Simulation Scope

The simulator checks functional correctness at the instruction level. It verifies the values written to registers and memory and whether control-flow instructions change the PC as expected. It does not model propagation delay, setup and hold time, clock skew, transistor behaviour, or physical wiring. A gate-level or HDL implementation would be a separate next step.

4.1.3 Reproducibility

The accompanying Python source file, 8bit_processor_sim.py, contains the instruction encoder, processor state model, demonstration program, and self-tests. Running the file prints the instruction trace and the result of each self-test. The simulator is included as supporting project material.

4.2 Hardware & Software Used

The project is a software-only functional simulation. No physical hardware, HDL simulator, or FPGA board was used; the processor behaviour is validated entirely by a reference software model.

- **Software:** Python 3, used to run the reference simulator source file 8bit_processor_sim.py.

- **Hardware:** any computer capable of running Python 3; no external hardware is required.

4.3 Testing Technology Used

Testing is carried out as instruction-level functional checks on the Python reference model. Each test initialises data memory, executes a short instruction sequence, and compares the observed output or processor state with the expected result. The six test cases and their outcomes are presented in Section 5.2.

4.4 Demonstration Program

The demonstration program shows basic data transfer and arithmetic. It first loads 5 into ACC and stores it at data address 0. It then loads 3, adds the value at address 0, and stores the sum at address 1. Finally, it reloads the sum, subtracts the original value 5, writes the result to OUT, and halts.

Table 4.1: Demonstration program

PC	Machine code	Assembly instruction	Purpose
0	0x15	LDI 5	Load decimal 5 into ACC.
1	0x30	STA 0	Store 5 at RAM[0].
2	0x13	LDI 3	Load decimal 3 into ACC.
3	0x40	ADD 0	Add RAM[0] (5) to ACC (3), giving 8.
4	0x31	STA 1	Store 8 at RAM[1].
5	0x21	LDA 1	Load 8 from RAM[1] into ACC.
6	0x50	SUB 0	Subtract RAM[0] (5), giving 3.
7	0xE0	OUT	Copy ACC (3) to OUT.
8	0xF0	HLT	Stop execution.

4.4.1 Expected Final State

Table 4.2: Expected final state of the processor

State element	Expected value	Explanation
ACC	0x03	Final result after 8 - 5.
RAM[0]	0x05	Original value stored at address 0.
RAM[1]	0x08	Sum stored at address 1.
OUT	0x03	Value captured by OUT.
Z	0	Final result is non-zero.
C	0	No carry or borrow in the final SUB operation.

CHAPTER 5
RESULT AND DISCUSSION

5.1 Simulation Trace and Results

The table below records the state after every executed instruction. PC shows the address from which the instruction was fetched. IR shows the encoded byte. ACC, RAM[0], RAM[1], OUT, Z, and C show the post-execution state.

Table 5.1: Simulation trace of the demonstration program

Step	PC	IR	Instruction	ACC	M[0]	M[1]	OUT	Z	C
1	00	15	LDI 5	05	00	00	00	0	0
2	01	30	STA 0	05	05	00	00	0	0
3	02	13	LDI 3	03	05	00	00	0	0
4	03	40	ADD 0	08	05	00	00	0	0
5	04	31	STA 1	08	05	08	00	0	0
6	05	21	LDA 1	08	05	08	00	0	0
7	06	50	SUB 0	03	05	08	00	0	0
8	07	E0	OUT	03	05	08	03	0	0
9	08	F0	HLT	03	05	08	03	0	0

5.1.1 Result Interpretation

After execution, RAM[0] contains 0x05 and RAM[1] contains 0x08. The final subtraction changes ACC from 0x08 to 0x03. OUT captures this value, and the HLT instruction stops the processor. The final output is decimal 3, represented as 00000011 in binary and 0x03 in hexadecimal.

5.1.2 Observation

The trace demonstrates that the processor can transfer values between ACC and data memory, perform an arithmetic operation using a memory operand, update the accumulator, transfer a result to the output register, and stop cleanly. The result is consistent with the intended program sequence.

5.2 Verification and Test Cases

In addition to the demonstration program, six independent functional checks were run against the reference model. Each test initialises data memory, executes a short instruction sequence, and compares the observed output or processor state with the expected result.

Table 5.2: Functional test cases and observed results

Test	Input / instruction sequence	Expected behaviour	Observed result
1. Addition	RAM[0]=5, RAM[1]=3; LDA 0, ADD 1, OUT	Output = 8	PASS - OUT=0x08
2. Carry	RAM[0]=250, RAM[1]=10; LDA 0, ADD 1, OUT	8-bit result = 4; C=1	PASS - OUT=0x04, C=1
3. Subtraction	RAM[0]=8, RAM[1]=5; LDA 0, SUB 1, OUT	Output = 3; borrow=0	PASS - OUT=0x03
4. AND	RAM[0]=0xAA, RAM[1]=0x0F; LDA 0, AND 1, OUT	0xAA AND 0x0F = 0x0A	PASS - OUT=0x0A
5. JZ branch	LDI 0, JZ 3, LDI 5, OUT	Zero condition branches to address 3	PASS - branch taken
6. Halt	LDI 7, HLT, LDI 2	Execution stops after HLT	PASS - 2 steps executed

5.2.1 Verification Summary

All six functional test cases passed in the provided Python reference model. These results support the correctness of the defined instruction semantics for the tested cases. They should not be interpreted as proof of gate-level timing, electrical behaviour, or physical hardware operation.

The source file prints the test outcomes when executed. The reported pass/fail results are reproducible using Python 3 and do not depend on an external HDL simulator.

5.3 Applications and Advantages

5.3.1 Educational Applications

- Demonstrating binary instruction encoding and the role of an instruction decoder.
- Showing how registers, program memory, data memory, and the ALU interact during program execution.
- Teaching arithmetic flags and conditional branching using small, traceable examples.
- Providing a starting point for laboratory extensions in Logisim, Verilog, VHDL, or FPGA tools.

5.3.2 Advantages of the Proposed Model

- Simple architecture: the accumulator-based datapath is easy to follow.
- Compact encoding: each instruction occupies one byte.
- Clear separation: program memory and data memory have distinct roles.
- Traceable execution: the small address space makes the state changes easy to inspect.

- Extensible structure: registers, addressing modes, instructions, and I/O can be added in later versions.

5.3.3 Practical Relevance

The design is a teaching model rather than a processor intended for production use. Its relevance is that it illustrates the same broad building blocks found in larger systems: instruction control, registers, memory interfaces, an ALU, status flags, and I/O.

CHAPTER 6
CONCLUSION AND FUTURE SCOPE

6.1 Conclusion

The project defined a compact 8-bit accumulator-based mini processor and demonstrated its functional behaviour using a reference simulator. The design includes an 8-bit datapath, a 4-bit program counter, separate 16 x 8-bit program and data memories, an instruction register, a control unit, an ALU, status flags, and an output register.

A fixed 8-bit instruction format was defined with a 4-bit opcode and a 4-bit operand. The instruction set supports data transfer, arithmetic, bitwise logic, branching, output, and halt operations. The fetch-decode-execute sequence explains how the processor moves from one instruction to the next and how the control unit coordinates the datapath.

The demonstration program stored the values 5 and 8 in data memory, calculated 8 - 5, and produced the final output 3 (0x03). Six additional functional checks covering addition, carry, subtraction, AND, conditional branching, and halt behaviour passed in the reference model.

The work meets the educational goal of illustrating the main components and operation of a small processor. A future version can add a gate-level schematic, HDL implementation, clock-accurate simulation, and FPGA testing to move from functional architecture to hardware validation.

6.2 Future Scope

6.2.1 Current Limitations

- The address space is small: only 16 instruction locations and 16 data locations are available.
- The 4-bit immediate field can directly represent only values from 0 to 15.
- The accumulator architecture has one main working register, which can require additional memory transfers.
- The current reference model is instruction-level and does not model real clock timing or hardware propagation delays.
- There is no stack, interrupt controller, pipelining, cache, or memory-mapped peripheral subsystem.
- The current output interface is represented by a register rather than a physical display or external device.

6.2.2 Possible Improvements

- Expand program and data memory and increase the address width.
- Add general-purpose registers and register-to-register instructions.
- Add immediate arithmetic, shift, compare, and stack instructions.
- Implement the design in Verilog or VHDL and verify it with a hardware description language testbench.
- Create a gate-level schematic in a digital logic simulator and inspect timing waveforms.
- Prototype the processor on an FPGA and connect LEDs, switches, or a seven-segment display.

6.2.3 Future Implementation Path

A suitable next phase is to translate the architectural specification into synthesizable HDL. The design can then be simulated at RTL, checked against the instruction trace in this report, synthesised, and tested on an FPGA development board. The same demonstration program can be reused as a hardware test case.

REFERENCES

[1] Mano, M. Morris. Computer System Architecture, 3rd Edition. Pearson, 1993.

[2] Mano, M. Morris, and Michael D. Ciletti. Digital Design, 5th Edition. Pearson, 2013.

[3] Stallings, William. Computer Organization and Architecture: Designing for Performance, 10th Edition. Pearson, 2016.

[4] Patterson, David A., and John L. Hennessy. Computer Organization and Design: The Hardware/Software Interface, RISC-V Edition, 2nd Edition. Morgan Kaufmann, 2020.

[5] Python Software Foundation. Python 3 Documentation. https://docs.python.org/3/ (accessed 13 November 2026).

Repository information

GitHub Repository: [INSERT YOUR REPOSITORY URL BEFORE SUBMISSION]

APPENDIX A
INSTRUCTION ENCODING AND SIMULATOR

A.1 Instruction Encoding Examples

Table A.1: Instruction encoding examples

Instruction	Binary encoding	Hex	Interpretation
LDI 5	0001 0101	0x15	Opcode 0001; immediate 0101.
STA 0	0011 0000	0x30	Opcode 0011; data address 0000.
ADD 0	0100 0000	0x40	Opcode 0100; add RAM[0] to ACC.
SUB 0	0101 0000	0x50	Opcode 0101; subtract RAM[0] from ACC.
OUT	1110 0000	0xE0	Opcode 1110; capture ACC in OUT.
HLT	1111 0000	0xF0	Opcode 1111; halt execution.

A.2 Run the Reference Simulator

Save 8bit_processor_sim.py in a folder with Python 3 installed. Run the following command from a terminal:

```
python 8bit_processor_sim.py
```

The program prints the demonstration trace, the final output, and the six functional self-test results. A successful run ends with the message “ALL TESTS PASSED”.

A.3 Source File Note

The included Python file is a functional reference model for the architecture specified in this report. It is intended to make the instruction behaviour reproducible. It is not a synthesizable HDL design and should not be described as an FPGA or gate-level implementation.